



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5

по дисциплине

«Разработка баз данных»

Выполнил студент группы ИНБО-20-23

Школьный И.В.

Проверил доцент

Исаева М.В.

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА №5. ОБЪЕКТЫ БАЗЫ ДАННЫХ: ПРЕДСТАВЛЕНИЯ И ХРАНИМЫЕ ПРОЦЕДУРЫ

Цель работы:

Работа направлена на формирование у студентов углубленных навыков работы с объектами баз данных в СУБД PostgreSQL, смещая акцент от прямого манипулирования данными к созданию переиспользуемых логических конструкций.

Постановка задачи:

Задание №1: создание модифицируемого представления

Для вашей базы данных создать простое модифицируемое представление, которое отбирает строки из одной таблицы по определенному критерию.

Задание №2: модификация данных через представление

Продemonстрировать возможность изменения данных в базовой таблице через представление, созданное в Задании №1. Для этого необходимо выполнить два запроса:

1. Добавить новую запись с помощью оператора INSERT.
2. Удалить существующую запись с помощью оператора DELETE.

Задание №3: создание немодифицируемого аналитического представления

Для вашей базы данных создать единое немодифицируемое представление для аналитических целей. Представление должно объединять данные как минимум из двух таблиц и содержать агрегирующие функции (COUNT, SUM, AVG и т.д.) и группировку (GROUP BY).

Задание №4: использование аналитического представления в запросах

Написать SELECT-запрос, который использует созданное в Задании №3 аналитическое представление в качестве источника данных для дальнейшей фильтрации или анализа.

Задание №5: Создание и обновление материализованного представления

1. Создать материализованное представление для ускорения выполнения ресурсоемкого аналитического запроса. Например, для БД «Аптека» можно создать представление, которое заранее рассчитывает общую сумму продаж для каждого покупателя.

2. Продемонстрировать процесс обновления данных в представлении с помощью команды `REFRESH MATERIALIZED VIEW viewName;`.

Задание №6: разработка пользовательской функции для аналитических вычислений

1. Разработать пользовательскую функцию, которая инкапсулирует комплексный аналитический расчет. Функция должна принимать на вход идентификатор (например, `manufacturer_id`) и возвращать одно скалярное значение (например, общую сумму продаж продукции данного производителя), вычисленное на основе соединения нескольких таблиц и применения агрегатных функций.

2. Продемонстрировать вызов функции в составе `SELECT`-запроса.

Задание №7: разработка хранимой процедуры для выполнения сложной операции

Разработайте хранимую процедуру, которая выполняет безопасную операцию по изменению данных. Процедура должна принимать на вход ID какой-либо записи и числовое значение (например, количество).

Внутри процедуры необходимо проверить, достаточно ли текущего значения в числовом поле одной таблицы для выполнения операции.

- Если да – уменьшите это значение и добавьте новую запись в другую, связанную таблицу.

- Если нет – операция должна полностью прерваться, не внося никаких изменений в данные. Для сообщения о результате используйте выходной параметр, который вернёт статус успеха или неудачи.

Задание №8: демонстрация вызова хранимой процедуры

Привести два примера вызова процедуры, созданной в Задании №7:

- Успешный вызов, который добавляет в вашу базу данных уникальную запись.

- Неудачный вызов, который демонстрирует срабатывание реализованной проверки целостности и возврат пользовательской ошибки.

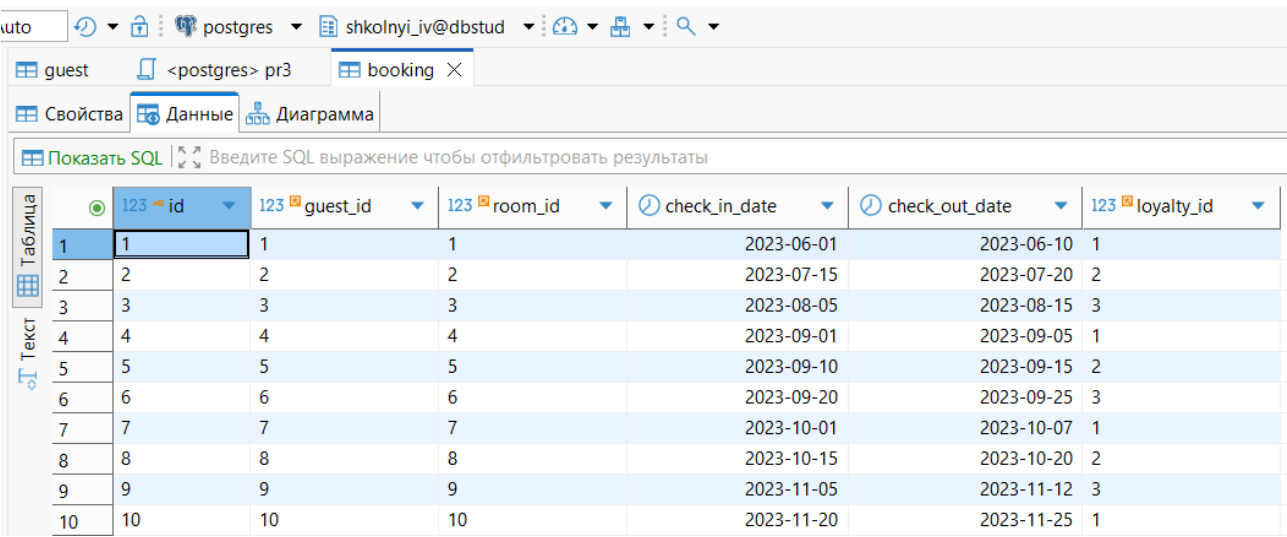
Каждый SQL-запрос сопровождать комментарием, объясняющим его назначение и логику работы с учетом специфики вашей базы данных.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

В предыдущих работах основной фокус был на извлечении и анализе данных с помощью сложных запросов, подзапросов и обобщенных табличных выражений. Эта работа делает следующий шаг: мы научимся инкапсулировать (прятать) эту сложность внутри специальных объектов базы данных — представлений и хранимых процедур. Это позволяет не только упростить код приложений, работающих с базой данных, но и перенести часть бизнес-логики непосредственно на уровень СУБД, повышая производительность, надежность и безопасность системы.

Ниже приводятся таблицы, используемые для построения запросов.

Таблица 1. Таблица Бронирование (Booking)



	123 id	123 guest_id	123 room_id	check_in_date	check_out_date	123 loyalty_id
1	1	1	1	2023-06-01	2023-06-10	1
2	2	2	2	2023-07-15	2023-07-20	2
3	3	3	3	2023-08-05	2023-08-15	3
4	4	4	4	2023-09-01	2023-09-05	1
5	5	5	5	2023-09-10	2023-09-15	2
6	6	6	6	2023-09-20	2023-09-25	3
7	7	7	7	2023-10-01	2023-10-07	1
8	8	8	8	2023-10-15	2023-10-20	2
9	9	9	9	2023-11-05	2023-11-12	3
10	10	10	10	2023-11-20	2023-11-25	1

Таблица 2. Таблица Документы о бронировании (Booking_document)

auto postgres shkolnyi_iv@dbstud

booking_document

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 booking_id	issue_date	A-Z status
1	1	1	2023-05-15	Confirmed
2	2	2	2023-06-20	Paid
3	3	3	2023-07-10	Completed
4	4	4	2023-08-15	Confirmed
5	5	5	2023-08-25	Paid
6	6	6	2023-09-10	Completed
7	7	7	2023-09-20	Confirmed
8	8	8	2023-10-05	Paid
9	9	9	2023-10-25	Completed
10	10	10	2023-11-10	Confirmed

Таблица 3. Таблица Брошюра (Brochure)

auto postgres shkolnyi_iv@dbstud

brochure

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 hotel_id	A-Z language	publication_date
1	1	1	Russian	2022-01-10
2	2	2	English	2022-03-15
3	3	3	German	2022-05-20
4	4	1	English	2022-02-28
5	5	2	German	2022-04-10
6	6	3	Russian	2022-06-15
7	7	1	German	2022-07-20
8	8	2	English	2022-08-25
9	9	3	English	2022-09-30
10	10	1	Russian	2022-10-05
11	11	2	Russian	2022-11-12
12	12	3	German	2022-12-18

Таблица 4. Таблица Регистрация (Check_in)

Auto postgres shkolnyi_iv@dbstud

check_in ×

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 guest_id	123 total_bill	123 room_id	123 doc_id
1	1	1	45 000	1	1
2	2	2	42 500	2	2
3	3	3	120 000	3	3
4	4	4	18 000	4	4
5	5	5	39 000	5	5
6	6	6	57 500	6	6
7	7	7	31 200	7	7
8	8	8	41 000	8	8
9	9	9	84 000	9	9
10	10	10	27 500	10	10

Таблица 5. Таблица Работник (Employee)

Auto postgres shkolnyi_iv@dbstud

employee ×

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	A-Z full_name	123 position_id	hire_date	123 hotel_id
1	1	Смирнова Ольга Дмитриевна	1	2018-06-10	1
2	2	Кузнецов Андрей Петрович	2	2020-02-15	2
3	3	Васильева Елена Ивановна	3	2021-05-20	3
4	4	Петров Максим Игоревич	4	2019-03-12	1
5	5	Семенова Ирина Васильевна	5	2020-08-25	2
6	6	Ковалев Александр Петрович	6	2018-11-30	3
7	7	Новикова Юлия Сергеевна	7	2021-02-14	1
8	8	Данилов Алексей Викторович	8	2017-07-22	2
9	9	Тихонова Марина Олеговна	9	2016-05-18	3
10	10	Фролов Денис Николаевич	10	2022-01-10	1

Таблица 6. Таблица Гость (Guest)

Auto postgres shkolnyi_iv@dbstud

guest X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z full_name	A-Z phone_number	A-Z email
1	1	Козлова Мария Игоревна	7911123456	kozlova@mail.ru
2	2	Николаев Дмитрий Олегович	7902876543	nikolaev@gmail.com
3	3	Федорова Екатерина Павловна	7922555777	fedorova@yandex.ru
4	4	Громов Сергей Александрович	7913987654	gromov@mail.ru
5	5	Волкова Анастасия Викторовна	7904765432	volkova@gmail.com
6	6	Белов Артем Сергеевич	7923876541	belov@yandex.ru
7	7	Соколова Ольга Николаевна	7915234567	sokolova@mail.ru
8	8	Иванов Иван Иванович	7910123456	ivanov@mail.ru
9	9	Петрова Анна Сергеевна	7909876543	petrova@gmail.com
10	10	Сидоров Алексей Викторович	7921555777	sidorov@yandex.ru

Таблица 7. Таблица Документы гостя (Guest_document)

uto postgres shkolnyi_iv@dbstud

guest_document X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	123 guest_id	A-Z document_data	A-Z document_type	issue_date	expiry_date
1	1	1	1234567890	Passport	2015-05-20	2025-05-20
2	2	2	0987654321	Birth certificate	2020-01-15	2030-01-15
3	3	3	1122334455	Driver license	2018-11-30	2028-11-30
4	4	4	4455667788	Passport	2017-03-10	2027-03-10
5	5	5	5566778899	Driver license	2019-08-22	2029-08-22
6	6	6	6677889900	Passport	2016-11-05	2026-11-05
7	7	7	7788990011	Birth certificate	2021-02-14	2031-02-14
8	8	8	8899001122	Driver license	2018-07-30	2028-07-30
9	9	9	9900112233	Passport	2019-12-20	2029-12-20
10	10	10	1011223344	Birth certificate	2020-04-18	2030-04-18

Таблица 8. Таблица Гостиница (Hotel)

uto postgres shkolnyi_iv@dbstud

guest_document hotel X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z name	A-Z address	123 star_rating	A-Z description
1	1	City Lights	ул. Невский проспект, 45	4	Современный отель в историческом центре
2	2	River Side	наб. реки Мойки, 12	5	Элитный отель с видом на реку
3	3	Park Inn	ул. Садовая, 78	3	Комфортабельный отель рядом с парком

Таблица 9. Таблица Программа лояльности (Loyalty_program)

auto postgres shkolnyi_iv@dbstud

loyalty_program

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z name	123 discount	A-Z description
1	Gold Member	15	Exclusive discounts and early check-in for gold members
2	Silver Member	10	Special offers and room upgrades for silver members
3	Bronze Member	5	Basic discounts for frequent guests

Таблица 10. Таблица Должность (Position)

auto postgres shkolnyi_iv@dbstud

loyalty_program position

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z title	123 base_rate
1	Менеджер	50 000
2	Портье	35 000
3	Горничная	30 000
4	Администратор	45 000
5	Шеф-повар	80 000
6	Официант	32 000
7	Охранник	38 000
8	Уборщик	28 000
9	Бухгалтер	55 000
10	Маркетолог	48 000
11	IT-специалист	60 000
12	Директор	100 000
13	Аниматор	35 000

Таблица 11. Таблица Номер (Room)

auto

postgres

shkolnyi_iv@dbstud

loyalty_program

position

room X

Свойства

Данные

Диаграмма

Показать SQL

Введите SQL выражение чтобы отфильтровать результаты

	123 id	A-Z room_number	A-Z type	123 price_per_night	123 hotel_id
1	1	101	Standard	5 000	1
2	2	202	Deluxe	8 500	2
3	3	301	Suite	12 000	3
4	4	102	Standard	4 500	1
5	5	103	Deluxe	7 500	1
6	6	201	Standard	4 800	2
7	7	202	Deluxe	7 800	2
8	8	203	Suite	11 500	2
9	9	301	Standard	5 200	3
10	10	302	Deluxe	8 200	3
11	11	303	Suite	12 500	3
12	12	401	Standard	5 500	1

Задание 1. Создание модифицируемого представления

Пусть, необходимо создать модифицируемое представление, которое выбирает все номера отеля City Lights. Поскольку у этого отеля идентификатор равен 1, надо прописать это в условии.

Листинг 1. Создание модифицируемого представления

```
--1. Создание модифицируемого представления
```

```
create or replace view city_lights_rooms as
```

```
select
```

```
    id,
```

```
    room_number,
```

```
    type,
```

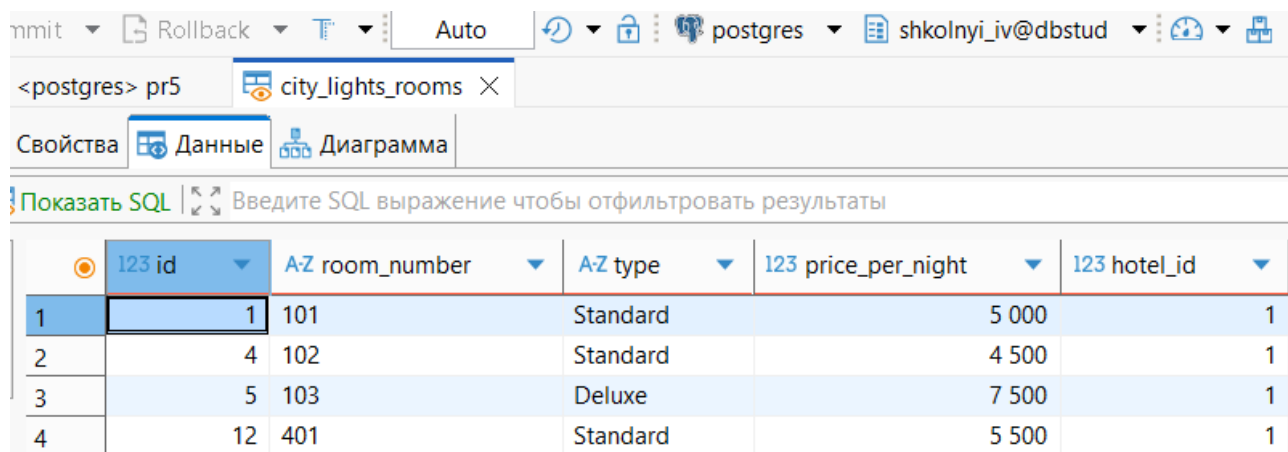
```
    price_per_night,
```

```
    hotel_id
```

```
from room
```

```
where hotel_id = 1;
```

Так выглядит результат.



	123 id	A-Z room_number	A-Z type	123 price_per_night	123 hotel_id
1	1	101	Standard	5 000	1
2	4	102	Standard	4 500	1
3	5	103	Deluxe	7 500	1
4	12	401	Standard	5 500	1

Рисунок 1 — Созданное модифицируемое представление

Задание 2. Модификация данных через представление

Идея заключается в том, что при изменении представления будет изменяться и базовая таблица. Добавим в представление новую запись. Отследим добавление в базовую таблицу и удалим запись.

Листинг 2. Модификация данных

```
-- 2.1 Добавление новой записи через представление
```

```
INSERT INTO city_lights_rooms (room_number, type, price_per_night, hotel_id)
VALUES ('105', 'Standard', 5200, 1);
```

-- Проверка добавления

```
SELECT * FROM room WHERE hotel_id = 1 AND room_number = '105';
```

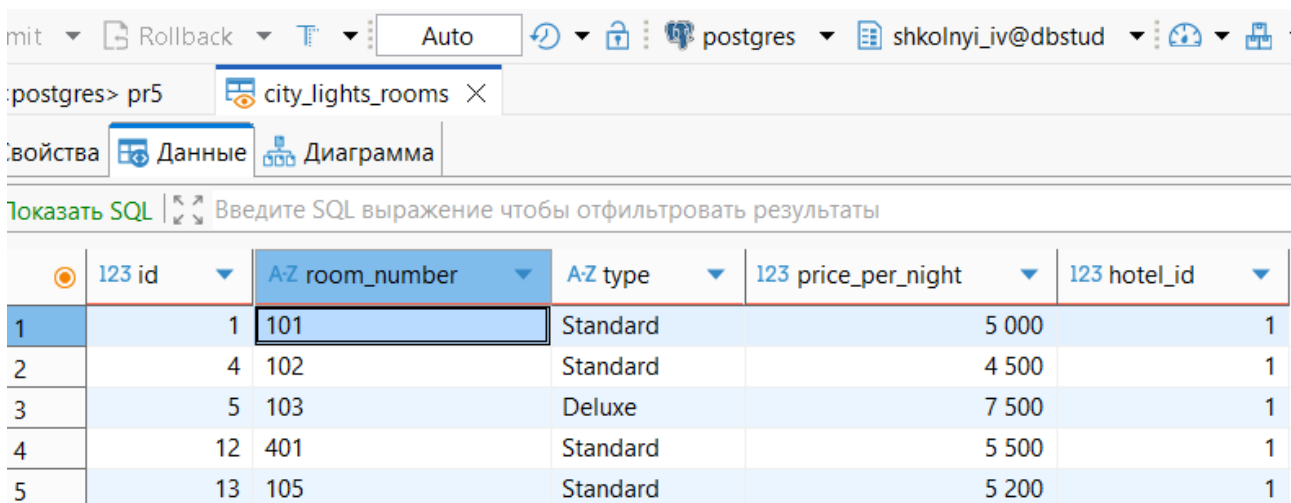
-- 2.2 Удаление записи через представление

```
DELETE FROM city_lights_rooms
WHERE room_number = '105';
```

-- Проверка удаления

```
SELECT * FROM room WHERE room_number = '105';
```

Так выглядят представление и таблица room.



	123 id	A-Z room_number	A-Z type	123 price_per_night	123 hotel_id
1	1	101	Standard	5 000	1
2	4	102	Standard	4 500	1
3	5	103	Deluxe	7 500	1
4	12	401	Standard	5 500	1
5	13	105	Standard	5 200	1

Рисунок 2 — Представление

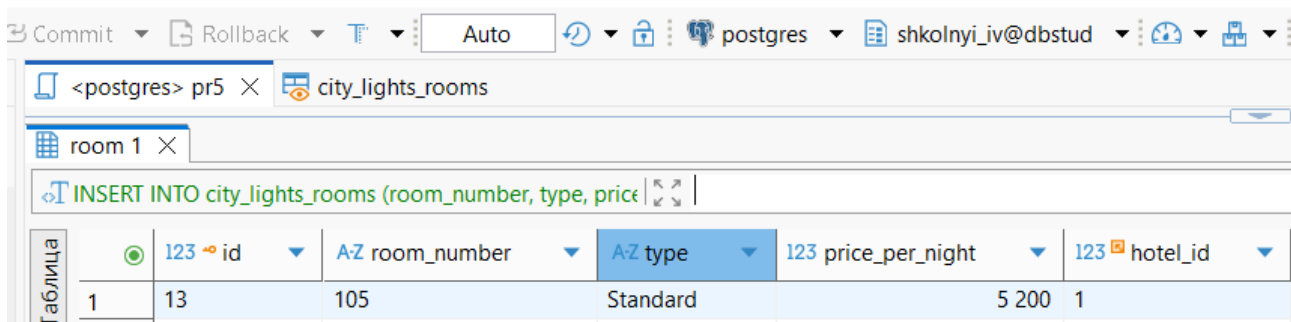


Таблица	123 id	A-Z room_number	A-Z type	123 price_per_night	123 hotel_id
1	13	105	Standard	5 200	1

Рисунок 3 — Таблица room

Как можем заметить, запись была успешно добавлена. Теперь наблюдаем ее удаление.

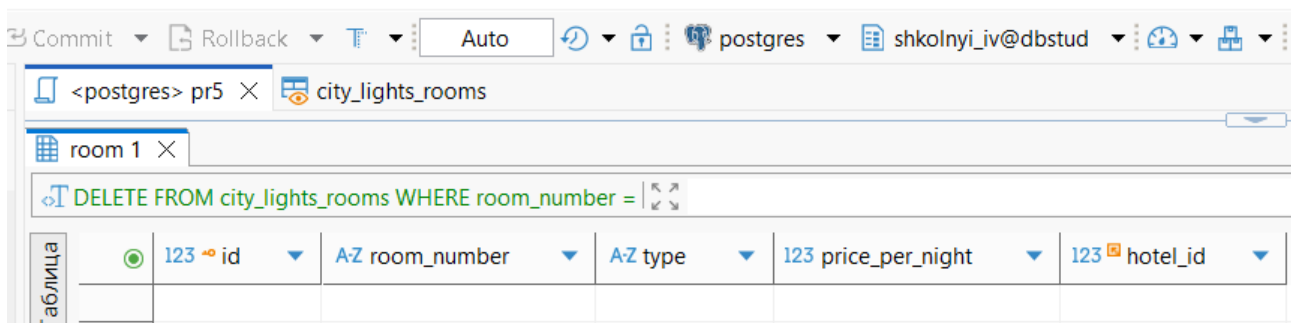


Рисунок 4 — Успешное удаление записи

Задание 3. Создание немодифицируемого аналитического представления

Отличия в этой представлении заключаются в том, что представление составляется из нескольких таблиц, используются агрегатные функции и группировка.

Листинг 3. Создание немодифицируемого представления

```
-- Аналитическое представление: статистика по отелям
CREATE OR REPLACE VIEW hotel_statistics AS
SELECT
    h.id AS hotel_id,
    h.name AS hotel_name,
    h.star_rating,
    COUNT(DISTINCT r.id) AS total_rooms,
    COUNT(DISTINCT e.id) AS total_employees,
    COUNT(DISTINCT b.id) AS total_bookings,
    COALESCE(SUM(ci.total_bill), 0) AS total_revenue,
    COALESCE(AVG(ci.total_bill), 0) AS avg_booking_value
FROM hotel h
LEFT JOIN room r ON h.id = r.hotel_id
LEFT JOIN employee e ON h.id = e.hotel_id
LEFT JOIN booking b ON r.id = b.room_id
LEFT JOIN check_in ci ON b.id = ci.room_id AND b.guest_id = ci.guest_id
GROUP BY h.id, h.name, h.star_rating;
```

Результат представлен ниже.

123 hotel_id	A-Z hotel_name	123 star_rating	123 total_rooms	123 total_employees	123 total_bookings	123 total_revenue	123 avg_booking_value
1	City Lights	4	4	4	3	408 000	34 000
2	River Side	5	4	3	4	516 600	43 050
3	Park Inn	3	4	3	3	694 500	77 166,6666666667

Рисунок 5 — Немодифицируемое представление

Задание 4. Использование аналитического представления в запросах

После создания немодифицируемого представления попробуем им воспользоваться. Профильтруем отели с высокой средней стоимостью бронирования.

Листинг 4. Использование немодифицируемого представления

```
-- Фильтрация отелей с высокой средней стоимостью бронирования
SELECT
    hotel_name,
    star_rating,
    total_rooms,
    total_revenue,
    avg_booking_value
FROM hotel_statistics
WHERE avg_booking_value > 40000
ORDER BY total_revenue DESC;
```

Результат представлен ниже.

	A-Z hotel_name	123 star_rating	123 total_rooms	123 total_revenue	123 avg_booking_value
1	Park Inn	3	4	694 500	77 166,6666666667
2	River Side	5	4	516 600	43 050

Рисунок 6 — Использование представления

Задание 5. Создание и обновление материализованного представления

Материализованное представление — это особый вид представления, который, в отличие от обычного, физически хранит результат своего запроса на диске. Это делает его похожим на реальную таблицу.

Основное преимущество: запросы к материализованному представлению выполняются мгновенно, так как СУБД не нужно каждый раз выполнять сложный базовый запрос. Это идеальное решение для «тяжёлых» аналитических отчетов, которые не требуют данных в реальном времени. Основной недостаток: данные в представлении могут устаревать. Для их обновления требуется специальная команда REFRESH MATERIALIZED VIEW. Рассмотрим пример создания материализованного представления.

Листинг 5. Создание материализованного представления

```
-- 5.1 Создание материализованного представления

CREATE MATERIALIZED VIEW monthly_revenue_report AS
SELECT
    h.name AS hotel_name,
    DATE_TRUNC('month', b.check_in_date) AS revenue_month,
    COUNT(b.id) AS booking_count,
    SUM(ci.total_bill) AS monthly_revenue
FROM hotel h
JOIN room r ON h.id = r.hotel_id
JOIN booking b ON r.id = b.room_id
JOIN check_in ci ON b.id = ci.room_id AND b.guest_id = ci.guest_id
GROUP BY h.name, DATE_TRUNC('month', b.check_in_date);

-- Проверка
SELECT * FROM monthly_revenue_report;
```

Результат представлен ниже.

	A-Z hotel_name	revenue_month	123 booking_count	123 monthly_revenue
1	City Lights	2023-06-01 00:00:00.000 +0300	1	45 000
2	City Lights	2023-09-01 00:00:00.000 +0300	2	57 000
3	Park Inn	2023-08-01 00:00:00.000 +0300	1	120 000
4	Park Inn	2023-11-01 00:00:00.000 +0300	2	111 500
5	River Side	2023-07-01 00:00:00.000 +0300	1	42 500
6	River Side	2023-09-01 00:00:00.000 +0300	1	57 500
7	River Side	2023-10-01 00:00:00.000 +0300	2	72 200

Рисунок 7 — Материализованное представление

Чтобы обновить представление, нужно воспользоваться командой `refresh materialized view <Имя представления>`.

Задание 6. Пользовательские функции

Это задание демонстрирует одно из главных преимуществ функций: инкапсуляцию сложного аналитического запроса. Вместо того чтобы каждый раз писать громоздкий запрос с несколькими соединениями и агрегациями, можно «спрятать» эту логику в функцию и вызывать её по простому имени. Это делает код приложения чище и снижает вероятность ошибок.

Листинг 6. Создание функции

```
-- Функция для расчета общего дохода по отелю
CREATE OR REPLACE FUNCTION get_hotel_total_revenue(p_hotel_id BIGINT)
RETURNS DECIMAL(12, 2) AS $$
DECLARE
    total_revenue DECIMAL(12, 2);
    hotel_exists BOOLEAN;
BEGIN
    -- Проверка существования отеля
    SELECT EXISTS(SELECT 1 FROM hotel WHERE id = p_hotel_id)
    INTO hotel_exists;

    IF NOT hotel_exists THEN
        RAISE EXCEPTION 'Отель с ID % не найден.', p_hotel_id;
```

```

END IF;

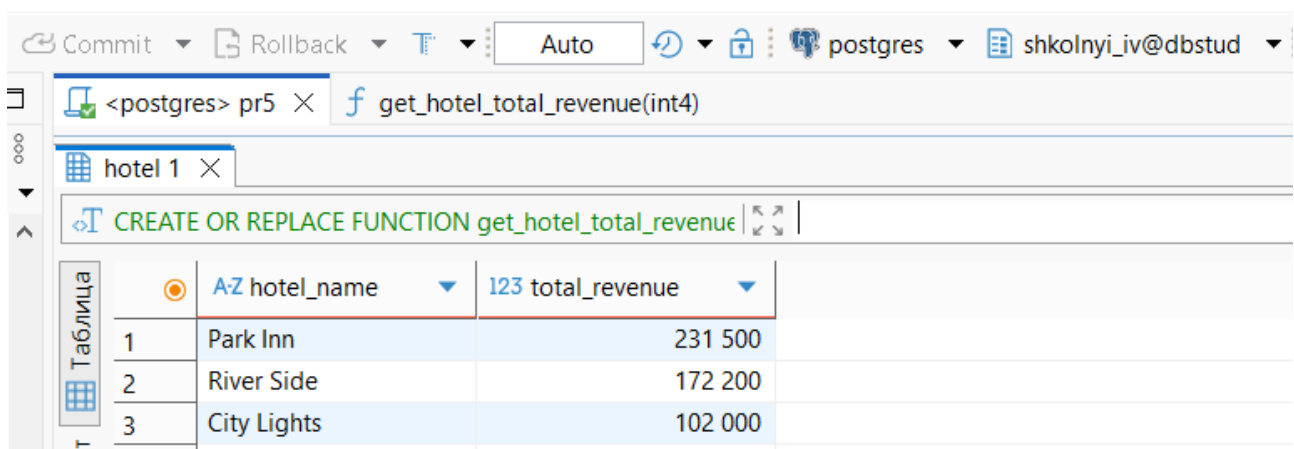
-- Расчет общего дохода
SELECT COALESCE(SUM(ci.total_bill), 0.00)
INTO total_revenue
FROM hotel h
JOIN room r ON h.id = r.hotel_id
JOIN booking b ON r.id = b.room_id
JOIN check_in ci ON b.id = ci.room_id AND b.guest_id = ci.guest_id
WHERE h.id = p_hotel_id;

RETURN total_revenue;
END;
$$ LANGUAGE plpgsql;

-- Демонстрация вызова функции
SELECT
    name AS hotel_name,
    get_hotel_total_revenue(id) AS total_revenue
FROM hotel
ORDER BY total_revenue DESC;

```

Результат вызова представлен ниже.



The screenshot shows a PostgreSQL client window with the following elements:

- Toolbar:** Includes buttons for Commit, Rollback, and Auto, along with icons for refresh, save, and connection management.
- Connection:** postgres, shkolnyi_iv@dbstud
- Query Editor:** Contains the SQL query: `CREATE OR REPLACE FUNCTION get_hotel_total_revenue`
- Results Panel:** Displays a table with 3 rows and 2 columns: `hotel_name` and `total_revenue`.

	A-Z hotel_name	123 total_revenue
1	Park Inn	231 500
2	River Side	172 200
3	City Lights	102 000

Рисунок 8 — Применение функции в запросе

Задание 7. Создание хранимой процедуры

В отличие от функции, процедура идеально подходит для реализации многошаговых бизнес-операций, которые изменяют состояние базы данных.

Листинг 7. Создание процедуры

```
-- Процедура для оформления бронирования с проверками
CREATE OR REPLACE PROCEDURE process_room_booking(
    p_room_id INT,
    p_guest_id INT,
    p_check_in_date DATE,
    p_check_out_date DATE,
    p_loyalty_id INT,
    OUT p_success BOOLEAN,
    OUT p_message TEXT
)
LANGUAGE plpgsql AS $$
DECLARE
    room_exists BOOLEAN;
    guest_exists BOOLEAN;
    loyalty_exists BOOLEAN;
    room_available BOOLEAN;
    new_booking_id INT;
BEGIN
    -- Проверка существования номера
    SELECT EXISTS(SELECT 1 FROM room WHERE id = p_room_id) INTO room_exists;
    IF NOT room_exists THEN
        p_success := FALSE;
        p_message := 'Ошибка: номер с ID ' || p_room_id || ' не найден.';
        RETURN;
    END IF;

    -- Проверка существования гостя
    SELECT EXISTS(SELECT 1 FROM guest WHERE id = p_guest_id) INTO guest_exists;
    IF NOT guest_exists THEN
        p_success := FALSE;
        p_message := 'Ошибка: гость с ID ' || p_guest_id || ' не найден.';
```

```

RETURN;

END IF;

-- Проверка существования программы лояльности
SELECT EXISTS(SELECT 1 FROM loyalty_program WHERE id = p_loyalty_id) INTO
loyalty_exists;

IF NOT loyalty_exists THEN
    p_success := FALSE;
    p_message := 'Ошибка: программа лояльности с ID ' || p_loyalty_id || ' не найдена.';
    RETURN;
END IF;

-- Проверка доступности номера на указанные даты
SELECT NOT EXISTS(
    SELECT 1 FROM booking
    WHERE room_id = p_room_id
    AND check_in_date <= p_check_out_date
    AND check_out_date >= p_check_in_date
) INTO room_available;

IF NOT room_available THEN
    p_success := FALSE;
    p_message := 'Ошибка: номер занят на указанные даты.';
    RETURN;
END IF;

-- Создание бронирования
INSERT INTO booking (guest_id, room_id, check_in_date, check_out_date, loyalty_id)
VALUES (p_guest_id, p_room_id, p_check_in_date, p_check_out_date, p_loyalty_id)
RETURNING id INTO new_booking_id;

-- Создание документа бронирования
INSERT INTO booking_document (booking_id, issue_date, status)
VALUES (new_booking_id, CURRENT_DATE, 'Confirmed');

```

```
p_success := TRUE;
```

```
p_message := 'Бронирование успешно оформлено. ID брони: ' || new_booking_id;
```

```
END;
```

```
$$;
```

Создание процедуры изображено ниже.

Commit ▾ Rollback ▾ T ▾ Auto ↺ 🔒 postgres ▾ shkolyi_iv@dbstud ▾	
*<postgres> pr5 ×	
Статистика 1 ×	
Name	Value
	p_message := 'Ошибка: гость с ID ' p_guest_id ' не найден.';
	RETURN;
	END IF;
	-- Проверка существования программы лояльности
	SELECT EXISTS(SELECT 1 FROM loyalty_program WHERE id = p_loyalty_id) INTO loyalty_exists;
	IF NOT loyalty_exists THEN
	p_success := FALSE;
	p_message := 'Ошибка: программа лояльности с ID ' p_loyalty_id ' не найдена.';
	RETURN;
	END IF;
	-- Проверка доступности номера на указанные даты
	SELECT NOT EXISTS(
	SELECT 1 FROM booking
	WHERE room_id = p_room_id
	AND check_in_date <= p_check_out_date
	AND check_out_date >= p_check_in_date
	) INTO room_available;
	IF NOT room_available THEN
	p_success := FALSE;
	p_message := 'Ошибка: номер занят на указанные даты.';
	RETURN;
	END IF;
	-- Создание бронирования
	INSERT INTO booking (guest_id, room_id, check_in_date, check_out_date, loyalty_id)
	VALUES (p_guest_id, p_room_id, p_check_in_date, p_check_out_date, p_loyalty_id)
	RETURNING id INTO new_booking_id;
	-- Создание документа бронирования
	INSERT INTO booking_document (booking_id, issue_date, status)
	VALUES (new_booking_id, CURRENT_DATE, 'Confirmed');
	p_success := TRUE;
	p_message := 'Бронирование успешно оформлено. ID брони: ' new_booking_id;
	END;
	\$\$

Рисунок 9 — Успешное создание процедуры

Задание 8. Демонстрация вызова хранимой процедуры

Выполним один успешный и один неудачный вызовы.

Листинг 6. Создание процедуры

```
-- 8.1 Успешный вызов
```

```
DO $$
```

DECLARE

v_success **BOOLEAN**;

v_message **TEXT**;

BEGIN

CALL *process_room_booking*(1, 1, '2024-01-15', '2024-01-20', 1, *v_success*, *v_message*);

RAISE NOTICE 'Результат: %', *v_message*;

END;

\$\$;

-- 8.2 Неудачный вызов (номер занят)

DO \$\$

DECLARE

v_success **BOOLEAN**;

v_message **TEXT**;

BEGIN

CALL *process_room_booking*(1, 2, '2023-06-05', '2023-06-12', 2, *v_success*, *v_message*);

RAISE NOTICE 'Результат: %', *v_message*;

END;

\$\$;

Результаты успешного и неудачного запросов:

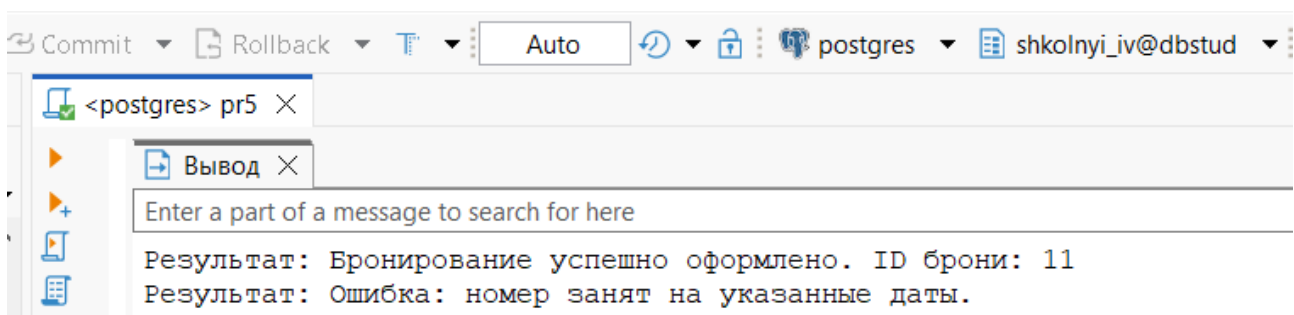


Рисунок 10 — Выполнение запросов

ВЫВОД

В ходе выполнения практической работы были освоены навыки создания переиспользуемых логических конструкций.